

API'DEN API'YE İLETİŞİM

NTP II – Hafta 11



API'DEN API'YE İLETİŞİM

Bu modülde, bir API'nin sadece veri sunmakla kalmayıp, aynı zamanda ****başka API'lerden veri tüketme**** yeteneğini inceleyeceğiz.

Bu yaklaşım, özellikle büyük ölçekli ve dağıtık uygulamalarda kritik öneme sahiptir (Microservice mimarileri).

Konu başlıklarımız:

1. Neden Harici Tüketim?
2. httpx Kütüphanesi ve Asenkronizm.
3. G/Ç (I/O) Bağlantılı İşlemlerde Performans Optimizasyonu.
4. Güvenlik: Ortam Değişkenleri ile Hassas Veri Yönetimi.



MODÜL ÖĞRENME HEDEFLERİ: MİKROSERVİS MANTIĞI

Öğrencilerin, API'lerin sadece 'sunucu' değil, aynı zamanda harici 'istemci' olabileceğini anlaması hedeflenmektedir.

Bu, modern yazılım mimarilerinin temelini oluşturur: ****Fonksiyonların servislere ayrılması**** (Microservice mimarileri).

Harici HTTP istekleri yapmak için Python'daki 'requests' ve 'httpx' kütüphanelerini ayırt etmeyi öğreneceğiz.

NEDEN HARİCİ VERİ TÜKETİMİ?

Bir API'nin, kendi iç veritabanına ek olarak dış kaynaklardan bilgi alması zorunludur:

****Veri Zenginleştirme (Data Enrichment):**** Müşteri verisine coğrafi konum eklemek veya bir IP adresinin tehdit puanını sorgulamak gibi.

****Teknoloji Ayrımı:**** Farklı servislerin farklı teknolojilerle yazılabilmesi.

****İş Yüğü Dağıtımı:**** Ağır iş yüklerini ana servisten ayırarak performansı korumak.



MİKROSERVİS MİMARİLERİNE GİRİŞ

Geleneksel Monolit uygulamalar, tüm işlevleri tek bir kod tabanında toplar. Ölçeklenmesi ve sürdürülmesi zordur.

Mikroservisler ise iş yeteneklerine göre küçük, bağımsız olarak konuşlandırılabilir hizmetlerdir.

Bu küçük servisler, işlevlerini yerine getirmek için birbirleriyle, genellikle HTTP (API) aracılığıyla iletişim kurarlar.

Harici veri tüketimi, bu mimarinin ****omurgasını**** oluşturur.

GİRİŞ/ÇIKIŞ (I/O) BAĞIMLI İŞLEMLER

Yazılımda iki ana darboğaz türü vardır:

****CPU-Bound (İşlemciye Bağlı):**** Ağır matematiksel hesaplamalar veya görüntü işleme gibi işlemler.

****I/O-Bound (Giriş/Çıkışa Bağlı):**** Veritabanı sorguları, dosya okuma/yazma veya harici API çağrıları gibi bekleme gerektiren işlemler.

Harici API çağrıları tamamen I/O-bound'dur, çünkü bekleme süresi ağ gecikmesine ve hedef sunucunun yanıt hızına bağlıdır.

PYTHON'DA HTTP İSTEKLERİ: REQUESTS

'requests' kütüphanesi, Python'da HTTP istekleri yapmak için en popüler ve kullanımı kolay kütüphanedir.

**** requests Senkrondur:**** Bir istek gönderildiğinde, yanıt gelene kadar kodun yürütülmesi durur ve sunucu kaynağı bloke olur.

Flask veya Django'nun geleneksel (WSGI) yapılarında bu kabul edilebilirken, modern asenkron çatılarda (ASGI/FastAPI) büyük bir performansı sorunudur.



NEDEN FASTAPI İÇİN REQUESTS KULLANILMAMALI?

FastAPI, **Asenkron Sunucu Geçidi Arayüzü (ASGI)** üzerine kurulmuştur. FastAPI asenkron olduğu için, 'requests' gibi senkron bir kütüphane kullanıldığında, tüm uygulama performansı bloke edilebilir. Hipotez: Eğer bir harici API çağrısı 1 saniye sürerse, senkron yapıda bu 1 saniye boyunca sunucu başka hiçbir isteği yanıtlayamaz. Bu 'bloklama' durumu, yüksek eşzamanlılık gerektiren uygulamalar için kabul edilemez.



ASYNCIO VE ASENKRON PROGRAMLAMA

Asenkron programlama, ****G/Ç beklemelerini verimli bir şekilde yönetmek**** için Python'a özgü bir paradigmaya dayanır.

Temel Yapılar: 'async def' ve 'await'.

'await', kodun çalıştığı coroutine'i (eş yordam) askıya alır. Bu sırada, olay döngüsü (event loop) başka bir göreve geçer.

****Sonuç:**** Sunucu, cevabı beklerken diğer kullanıcılara hizmet vermeye devam eder, bu da performans için devrim niteliğindedir.

ASYNC/AWAIT MANTIĞI

'async def', bir fonksiyonu Coroutine olarak işaretler. Bu, bu fonksiyonun askıya alınabilir olduğu anlamına gelir.

'await' kelimesi yalnızca 'async def' içinde kullanılabilir.

Bir işlem 'await' ile çağrıldığında (örn. 'await client.get(...)'), Python bu işlemi ağa gönderir ve yanıtı beklemek yerine ****kontrolü olay döngüsüne geri verir****. Olay döngüsü, bu sırada tamamlanmaya hazır başka görev olup olmadığını kontrol eder.

HTTXX KÜTÜPHANESİNİN TANITIMI

'httpx', Python için modern bir HTTP istemcisidir.
En önemli özelliği, hem ****senkron hem de asenkron**** modları desteklemesidir.
FastAPI gibi ASGI çatılarında en iyi performansı sağlamak için
'httpx.AsyncClient' kullanılır.
Kurulumu basittir: 'pip install httpx'.

ASYNC CLIENT İLE GÜVENLİ BAĞLANTI

Harici bağlantılar bir kaynaktır. Kaynakların açılıp kapatılması önemlidir. 'httpx.AsyncClient()' genellikle 'async with' bloğu içinde kullanılır. 'async with httpx.AsyncClient() as client:' yapısı, isteği gönderdikten sonra istemcinin bağlantılarını otomatik olarak düzgün bir şekilde kapatılmasını sağlar.

Senaryo: Analistler, bir IP veya domain eklerken bunların 'kötü şöhretli' (malicious) olup olmadığını manuel kontrol etmek zorunda kalıyor.

İstenen Çözüm: Kullanıcı yeni bir Gösterge (IoC) POST ettiğinde, API'miz arka planda halka açık bir tehdit API'sine GET isteği yapmalıdır.

Dönen bilgiyi (örn. coğrafi konum, sahip) alıp, kendi veritabanımızdaki gösterge modelinin 'description' alanına kaydetmeliyiz.

SENARYO 1: TEST VE ALAN KULLANIMI

Gerçek bir tehdit API'si (örn. AbuseIPDB) genellikle API anahtarı gerektirir. Ders amaçlı test için jsonplaceholder.typicode.com/users/1 gibi ücretsiz ve halka açık bir test API'si kullanılabilir. Bu test API'si, harici çağrı mantığını ve JSON yanıtını ayrıştırmayı öğrenmek için idealdir.

SENARYO 2: COĞRAFİ KONUM API'si

Başka bir örnek olarak, bir IP adresinin coğrafi konumunu öğrenmek için 'https://ip-api.com/' gibi ücretsiz bir coğrafi konum API'si kullanılabilir. Bu, bir IP'nin hangi ülkeden geldiği bilgisini alıp, Indicator modeline kaydetme pratiği sunar.

KODLAMA SÜRECİ ADIM 1: BAĞIMLILIKLAR

FastAPI projemizde harici istekler yapmaya başlamadan önce 'httpx' paketini kurmalıyız.

Kurulum komutu:

```
'pip install httpx'
```

Bu adım, ders akışında yaklaşık 15 dakika süren temel hazırlık aşamasıdır.

KODLAMA SÜRECİ ADIM 2: GEREKLİ İMPORTLAR

'main.py' dosyamıza 'httpx' kütüphanesini dahil etmemiz gerekiyor.

'import httpx # Yeni import'

Ayrıca FastAPI ve veritabanı işlemlerini yöneten temel kütüphaneler (FastAPI, Depends, HTTPException, Session) yerinde olmalıdır.



KODLAMA SÜRECİ ADIM 3: ENDPOINT'İN ASENKRON YAPILMASI

Harici API çağrısı yapacak olan 'create_indicator' endpoint'inin senkron dan asenkron e dönüştürülmesi zorunludur.

Fonksiyon tanımını 'def' yerine 'async def' olarak güncellemeliyiz:

```
'async def create_indicator(ioc: schemas.ioCCreateSchema, db: Session = Depends(get_db)):'
```

Bu değişiklik, FastAPI'nin asenkron doğasına uyum sağlar.

KODLAMA SÜRECİ ADIM 4: VERİ MODELİNİN GÜNCELLENMESİ

Müşteri talebine göre, harici API'den gelen veriyi saklamak için Indicator modeline bir alan eklenmelidir.

Bu alan genellikle 'description' (açıklama) olarak adlandırılır.

Veritabanı modeline (models.py): 'description = Column(String, nullable=True)' eklenmelidir.

'nullable=True' ifadesi, bu alanın boş bırakılabileceği anlamına gelir.

KODLAMA SÜRECİ ADIM 5: ŞARTLI SORGULAMA

Harici sorgulama, yalnızca belirli koşullar altında yapılmalıdır (Müşteri Talebi). Senaryomuzda bu koşul, göstergenin tipinin 'ip' olmasıdır:

```
'if ioc.type == 'ip':
```

Eğer koşul sağlanmazsa, varsayılan bir açıklama ('description = 'Yeni oluşturuldu.') kullanılır.

KODLAMA SÜRECİ ADIM 6: HARİCİ API URL TANIMLAMA

Kullanılacak test API'sinin URL'si tanımlanır:

'TEST_API_URL = fhttps://jsonplaceholder.typicode.com/users/1'

Gerçek projelerde bu URL'ler, yapılandırma dosyalarından veya ortam değişkenlerinden alınmalıdır.



KODLAMA SÜRECİ ADIM 7: ASENKRON İSTEMCİYİ BAŞLATMA

İstemci, 'async with' bloğu içinde başlatılır:

'async with httpx.AsyncClient() as client:'

Bu, bağlantı havuzunu (connection pool) yönetir ve işlemi tamamlandığında kaynağı serbest bırakır.

KODLAMA SÜRECİ ADIM 8: ASENKRON GET İSTEĞİ

İstek, 'await' anahtar kelimesi kullanılarak yapılır:

```
'response = await client.get(TEST_API_URL)'
```

Bu, 'client.get' işlemi tamamlanana kadar Coroutine'i askıya alır ve CPU'nun başka işlerle ilgilenmesini sağlar.

HATA YÖNETİMİ: HTTP STATUS KONTROLÜ

Harici API'ler her zaman başarılı yanıt vermez. Yanlış URL, yetkilendirme hatası (401/403) veya sunucu hatası (5xx) olabilir.

'`response.raise_for_status()`' fonksiyonu, yanıt HTTP durum kodu 4xx veya 5xx ise otomatik olarak bir '`HTTPStatusError`' istisnası fırlatır.

Bu, kodu basitleştirir ve başarısız isteklerin sessizce göz ardı edilmesini önler.

KODLAMA SÜRECİ ADIM 9: JSON PARŞİNG

Başarılı bir HTTP isteğinden sonra, yanıtın gövdesi JSON formatındadır. Bu veriyi Python sözlüğüne çevirmek için 'response.json()' metodu kullanılır. 'api_data = response.json()'

Bu sözlük yapısından ihtiyacımız olan bilgiyi ('name' veya 'country' gibi) almalıyız.

KODLAMA SÜRECİ ADIM 10: VERİYİ ÇIKARMA VE BİÇİMLENDİRME

Harici API'den alınan veriyi, kullanıcıya anlamlı gelecek şekilde formatlamalıyız.

Örnek:

'api_data.get('name')' ile kullanıcı adını alıyoruz.

'description = fHarici API'den alınan bilgi: {api_data.get('name')}'

Eğer API'den veri alınamazsa, 'get()' metodu hata vermeden 'None' döner.

HATA YÖNETİMİ: BAĞLANTI HATALARI

Harici API çağrıları doğası gereği güvensizdir (ağ sorunları, sunucu kesintisi).
Hata yakalama zorunludur.

'try...except httpx.RequestError as exc:'

Bu, DNS çözümleme hataları, zaman aşımı (timeout) veya bağlantı reddi gibi tüm ağ düzeyindeki sorunları yakalar.

HATA YÖNETİMİ: BAĞLANTI HATASI DURUMU

Eğer harici API'ye bağlanırken hata oluşursa (RequestError), sistemin çökmesini engellemeli ve bu durumu kaydetmeliyiz.

Örnek kodlama:

'print(fHarici API'ye bağlanırken hata oluştu: {exc})' (Loglama)

'description = Harici API sorgulanamadı.' (Kullanıcıya bilgi verme)

Bu durumda, loC yine de veritabanına kaydedilir, ancak açıklama alanı uyarı içerir.

VERİTABANI KAYIT SÜRECİ

API'den alınan (veya varsayılan) 'description' alanını kullanarak veritabanı modelini oluştururuz.

****Pydantic'ten SQLAlchemy'ye Çeviri:****

```
'db_indicator = models.Indicator(**ioc.model_dump(),  
description=description)'
```

'model_dump()' metodu, Pydantic şemasındaki tüm verileri sözlük olarak alır.

VERİTABANI İŞLEMLERİ

Oluşturulan veritabanı nesnesini oturuma ekleriz: 'db.add(db_indicator)'.
Değişiklikleri kalıcı hale getiririz: 'db.commit()'.
Veritabanı tarafından oluşturulan ID'ler gibi bilgileri almak için nesneyi yenileriz: 'db.refresh(db_indicator)'.
Son olarak, oluşturulan nesneyi API yanıtı olarak döndürürüz: 'return db_indicator'.

SENKRON BLOKLAMAYA KARŞI ASENKRON AKIŞ

WSGI (Flask/Django Senkron): İstek gelir -> Ağ isteği gönderilir -> 1 saniye beklenir (Sunucu bloke) -> Yanıt gelir.

ASGI (FastAPI Asenkron): İstek gelir -> Ağ isteği gönderilir -> Kontrol olay döngüsüne geçer -> Yeni istekler işlenir -> Yanıt gelince askıdaki göreve dönülür.

Bu sayede FastAPI, aynı anda 50 kullanıcıya hizmet vermeye devam edebilirken, senkron yapı tek bir kullanıcıyı bekler.



Varsayılan olarak, harici istekler sonsuza kadar bekleyebilir. Bu nedenle 'timeout' ayarı kritik öneme sahiptir. 'httpx.AsyncClient(timeout=5.0)' ile 5 saniyelik bir zaman aşımı belirlenebilir. Gerçek dünya uygulamalarında, başarısız olan istekler için yeniden deneme (retry) mekanizmaları uygulamak (Exponential Backoff ile) uygulamanın sağlamlığını artırır.

API ANAHTARI GÜVENLİĞİ: GİRİŞ

Gerçek tehdit istihbaratı API'leri (örn. VirusTotal, AbuseIPDB) genellikle kimlik doğrulama için bir API Anahtarı (API Key) gerektirir.

Bu anahtarlar, ödeme yapılan servis erişimini temsil eder ve son derece hassastır.

Bu hassas verileri ****asla doğrudan kodun içine (main.py'ye) yazmamalıyız****.

KİMLİK BİLGİSİ SIZINTISI (CREDENTIAL LEAK)

Bir API anahtarının doğrudan koda yazılması ve ardından kodun 'git' veya benzeri bir versiyon kontrol sistemine yüklenmesi, ****credential leak**** olarak bilinen tehlikeli bir zafiyettir.

Saldırganlar, bu sizin anahtarları kullanarak ücretli servis hesabınıza erişebilir veya kötü niyetli işlemler gerçekleştirebilir.

Bu zafiyet, siber güvenlikte en yaygın görülen ve kritik risklerden biridir.



GÜVENLİ YÖNTEM: ORTAM DEĞİŞKENLERİ

API anahtarları gibi hassas verilerin yönetimi için en iyi pratik, bunları ****Ortam Değişkenleri (Environment Variables)**** olarak yönetmektir.

****Avantajları:****

1. Koddan tamamen ayrılırlar.
2. 'git'e yüklenmezler.
3. Farklı ortamlarda (Geliştirme, Test, Üretim) farklı değerler kolayca kullanılabilir.

PYTHON'DA ORTAM DEĞİŞKENLERİNİ OKUMA

Python'da ortam değişkenleri 'os.environ' modülü ile okunabilir. FastAPI ekosisteminde genellikle Pydantic'in 'Settings' yönetimi kullanılır. Bu, '.env' dosyalarını da okuma yeteneği sunar ve değişkenleri tip güvenli hale getirir.



UYGULAMA ALANI: GOOGLE COLAB

Öğrencilerin ana PyCharm/Visual Studio Code projelerini bozmadan denemeler yapmaları için Google Colab gibi 'sandbox' ortamları kullanılır. Bu ortamlar, hızlı API testi ve alıştırılmaları için idealdir. Burada asenkron kodlama yerine, temel senkron 'requests' kütüphanesi pratik edilir.



COLAB ALIŞTIRMASI: TEMEL GET İSTEKLERİ

Öğrenciler, halka açık farklı API'leri (örneğin, 'https://catfact.ninja/fact' veya 'https://api.publicapis.org/entries') sorgulamayı öğrenir.

Bu alıştırmanın temel hedefleri:

1. Bir API'ye GET isteği yapma.
2. 'response.status_code' ile HTTP durumunu kontrol etme.
3. 'response.json()' ile dönen JSON verisini ayrıştırma.

SONUÇ: İLERİ SEVİYE API MÜHENDİSLİĞİ

API'ler artık izole servisler değildir; birbirleriyle konuşan karmaşık sistemlerdir. FastAPI'nin ASGI doğası, I/O-bound işlemler için 'httpx' ve 'async/await' kullanımını en iyi pratik haline getirmiştir.

Hata yönetimi ('try/except', 'raise_for_status') uygulamaların sağlamlığı için hayati önem taşır.

Siber güvenlik açısından, API anahtarlarının ortam değişkenleri ile yönetilmesi zorunludur.



EK DETAY: HTTP STATUS KODLARI SINIFLANDIRMASI

1xx: Bilgilendirme (Informational)

2xx: Başarılı (Success) – Örn: 200 OK, 201 Created.

3xx: Yönlendirme (Redirection)

4xx: İstemci Hatası (Client Error) – Örn: 404 Not Found, 403 Forbidden, 401 Unauthorized.

5xx: Sunucu Hatası (Server Error) – Örn: 500 Internal Server Error.

'httpx.raise_for_status()' 4xx ve 5xx durumlarında hata fırlatır.



EK DETAY: HTTP İSTEMCİ TÜRLERİ

Her istek için yeni bir TCP bağlantısı açıp kapatmak, performansı düşürür (overhead). Bu yüzden persistent connection (kalıcı bağlantı) kullanılır. 'httpx.AsyncClient' bağlantı havuzlama (connection pooling) mekanizmasını otomatik olarak kullanır.

Bu, aynı hedefe yapılan tekrar eden isteklerde bağlantı açma/kapama süresini ortadan kaldırarak performansı artırır.

SENKRON KODUN ASENKRON İÇİNDE KULLANIMI: UYARI

Eğer bir FastAPI endpoint'inde zorunlu olarak senkron kod (örn. 'requests' veya CPU-bound işlem) kullanmanız gerekirse, bunu doğrudan yapmamalısınız. FastAPI'nin bunu bir iş parçacığı havuzunda (thread pool) çalıştırmasını sağlamak için 'run_in_executor' kullanılabilir. Ancak, bu çözüm I/O bloklamasını tamamen ortadan kaldırmaz, sadece ana olay döngüsünden ayırır.

Sadece ortam değişkenleri yetmez. Kurumsal uygulamalarda profesyonel gizli bilgi yönetimi (Secret Management) araçları kullanılır:

1. AWS Secrets Manager
2. HashiCorp Vault
3. Azure Key Vault

Bu sistemler, sırları merkezi olarak saklar ve uygulamaların yalnızca yetkili olduklarında erişmesini sağlar.

COLAB ÖRNEĞİ: TEMEL JSON AYRIŞTIRMA

API'den dönen veri genellikle iç içe geçmiş sözlük ve liste yapılarından oluşur. Öğrenciler, Python'un sözlük erişim yöntemlerini kullanarak (köşeli parantez '[']' veya '.get()') karmaşık JSON yapıları içinden belirli bir alanı çekme pratiği yapmalıdır.

Örnek: `'data['results']['name']'`



FASTAPI'DE DEPENDS VE ASYNC

FastAPI'de 'Depends()' kullanılarak çağrılan bağımlılıklar (örneğin 'get_db'), asenkron veya senkron olabilir.

Eğer bağımlılıklar senkronsa, FastAPI bunları otomatik olarak ayrı bir iş parçacığı havuzunda çalıştırarak ana döngüyü korur.

Ancak endpoint'in kendisi ('create_indicator') harici I/O içeriyorsa, mutlaka 'async def' olmalıdır.



HTTXX VS. REQUESTS: ÖZET KARŞILAŞTIRMA

Requests:

- Senkron (Bloklayıcı)
- Basit kullanım
- Çoğu Django/Flask senaryosu için yeterli

httpx:

- Hem Senkron hem Asenkron destekli
- FastAPI ve ASGI için en iyi pratik
- Modern Python standartlarına daha uyumlu

API'DEN DÖNEN VERİ KONTROLÜ

Harici API'lerden gelen veriler her zaman beklediğimiz formatta olmayabilir. Bu nedenle, alınan 'api_data' nesnesinin içindeki alanları kontrol etmek (validation) kritik öneme sahiptir.

****Pydantic:**** Gelen harici veriyi doğrulamak için geçici bir Pydantic şeması kullanılabilir, bu da veri tip güvenliğini sağlar.



GELİŞMİŞ HATA YÖNETİMİ: TIMEOUT VE RETRY

Ağ gecikmeleri veya API'nin kısa süreli kesintileri sıkça yaşanır. Basit bir 'try/except' bloğu yeterli değildir. Dayanıklı sistemler, isteği belirli bir süre (örneğin 3 kez) ve artan bekleme süreleriyle (exponential backoff) yeniden denerler.

HTTPAUTH VE BAŞLIKLAR (HEADERS)

Çoğu harici API, kimlik doğrulama için API anahtarlarını şu yollarla ister:

1. Sorgu Parametresi (Query Parameter): URL'ye eklenir.
2. HTTP Başlığı (Header): Genellikle 'Authorization: Bearer <token>' formatında. 'httpx' ile başlık eklemek kolaydır: 'headers = {'X-API-Key': api_key}'.



HARİCİ API ENTEGRASYONUNDA SÜRÜM YÖNETİMİ

Tükettiğiniz harici API'ler zamanla değişecektir.
Bu değişikliklerden etkilenmemek için genellikle API'nin sürüm numarasını kullanmak iyi bir uygulamadır (örn. '/api/v1/users').
Eğer bir API sürüm yönetimini desteklemiyorsa, entegrasyonlarınızın beklenmedik şekilde bozulma riski yüksektir.

ASYNCIO PRENSİBİ: EŞZAMANLILIK (CONCURRENCY)

Python'daki 'asyncio', kodun aynı anda çalıştığı (paralellik) anlamına gelmez. ****Eşzamanlılık (Concurrency):**** Tek bir iş parçacığında (thread), görevlerin bekleme süresini (I/O) kullanarak hızlıca görevler arasında geçiş yapmaktır. Bu, tek bir garsonun (event loop), bir siparişi beklerken diğer masaya hizmet etmesine benzer.

KOD BLOKU İNCELEMESİ: ASYNC WITH

ASYNC bağlam yöneticisi ('async with'), '__aenter__' ve '__aexit__' asenkron metodlarını kullanır.

Bu, 'httpx.AsyncClient'in bloğa girerken güvenli bir şekilde başlatılmasını ve bloktan çıkarken asenkron olarak kaynakları serbest bırakmasını sağlar.

Bu manuel bağlantı kapatma gerekliliğini ortadan kaldırır ve hata durumlarında bile temizliği garantiler.

API'den API'ye çağrılar, özellikle yoğun veri işleme gerekiyorsa, kullanıcı isteği süresini uzatabilir.

Eğer harici API çağrısı, veritabanına kaydetmeden önce büyük bir hesaplama gerektiriyorsa, bu işlem Arka Plan Görevine (Background Task) taşınmalıdır. FastAPI, bu tür uzun süren işlemleri yönetmek için kendi 'BackgroundTasks' mekanizmasına sahiptir.

KAYNAK YÖNETİMİ: İPTAL SİNYALLERİ

Asenkron uygulamalarda, bir görevin (task) çok uzun sürdüğünü veya artık gerekli olmadığını anladığımızda onu iptal etmek önemlidir. 'asyncio' ve 'httpx', görev iptallerini destekler. Eğer bir kullanıcı API isteğini iptal ederse, FastAPI bu bilgiyi 'httpx'e ileterek harici ağ isteğinin de zamanından önce sonlandırılmasını sağlayabilir.



Genellikle SQLAlchemy'nin standart ORM oturumları (Session), senkron olarak çalışır (bloklayıcıdır).

Bu nedenle 'db.commit()' ve 'db.add()' işlemleri teknik olarak senkrondur, ancak FastAPI bunları ayrı bir iş parçacığı havuzunda yürüterek ana döngüyü korur.

Modern uygulamalar için, asenkron veritabanı sürücülerini (örn. SQLAlchemy 2.0+ AsyncEngine) kullanmak daha iyi bir çözümdür.



Bir HTTP yanıtının durumu, isteğin başarılı olup olmadığını gösterir. Colab alıştırmasında öğrenciler, gelen yanıtın 200 olup olmadığını kontrol etmeli ve duruma göre farklı çıktı vermeyi öğrenmelidirler.

Örnek: `'if response.status_code == 200: print('Başarılı') else: print('Hata kodu:', response.status_code)'`

COLAB PRATİĞİ: HEADER İNCELEMESİ

HTTP yanıtları sadece içerikten (body) oluşmaz, aynı zamanda başlıklar (headers) da içerir.

Başlıklar, önbelleğe alma (caching), içerik tipi (Content-Type) ve sunucu bilgileri gibi önemli meta veriler sağlar.

Öğrenciler, 'response.headers' sözlüğünü inceleyerek yanıtın Content-Type'inin 'application/json' olup olmadığını kontrol edebilir.

Üretim ortamında, harici API çağrılarının başarılı mı yoksa başarısız mı olduğunu bilmek zorundayız.

'print()' kullanmak yerine, profesyonel bir loglama çerçevesi (örn. Python's 'logging' module) kullanılmalıdır.

Başarısız çağrılarda, hatanın türü ve harici API'nin verdiği yanıt kodları loglanmalıdır.



MİMARİ DÜŞÜNCE: GATEWAY VE AGGREGATION

FastAPI'miz bu senaryoda bir API Gateway (geçit) işlevi görüyor. Kullanıcıdan bir talep alır, bu talebi tamamlamak için birden fazla harici API'ye (tehdit, coğrafi konum, sahip bilgisi) eş zamanlı çağrı yapar. Tüm bu verileri toplayıp (aggregation) tek, zenginleştirilmiş bir yanıt olarak geri döndürür.



API GÜVENLİĞİ: RATE LIMITING

Tükettiğimiz harici API'lerin çoğu, hizmetlerini kötüye kullanmamızı engellemek için hız sınırlaması (rate limiting) uygular.

Eğer API'miz çok hızlı bir şekilde çok sayıda harici çağrı yaparsa, 429 Too Many Requests hatası alırız.

Bu durumu yönetmek için kendi API'mizde harici isteklere giden hız sınırlamasını uygulamamız gerekir.

KODLAMA İPUCU: GET İSTEKLERİNDE PARAMETRELER

API'lere veri gönderirken sorgu parametreleri (query parameters) sıklıkla kullanılır.

'httpx'te bu, URL'yi manuel olarak biçimlendirmek yerine 'params' argümanı kullanılarak güvenli bir şekilde yapılır.

Örnek: `'await client.get(url, params={'ip': ioc_value, 'key': api_key})'`

Bu, karakter kodlama hatalarını önler.



GÖSTERGE MODELİ GENİŞLEMESİ

IoC modeli sadece 'description' değil, harici API'den gelen veriyi ayrı alanlarda da tutabilir.

Örneğin, 'country' (Ülke) veya 'threat_score' (Tehdit Puanı) gibi spesifik alanlar eklenerek veritabanı daha yapılandırılmış hale getirilebilir.

Bu, arama ve filtreleme yeteneklerini artırır.



ASYNc CLIENT VE İŞ PARÇACIKLARI

'httpx.AsyncClient' tamamen tek bir iş parçacığı (thread) üzerinde çalışır. Bekleme sırasında kontrolü serbest bırakması, işletim sisteminin G/Ç kaynaklarının boş kalmasını engeller. Bu, geleneksel çoklu iş parçacığı (multi-threading) yerine, asenkron tek iş parçacığı modelinin ağ uygulamaları için daha hafif ve verimli olmasını sağlar.

PYTHON SÜRÜM UYUMLULUĞU

'async' ve 'await' anahtar kelimeleri Python 3.5 sürümünde tanıtılmıştır. FastAPI ve 'httpx' gibi modern ASGI kütüphaneleri bu yapıyı kullandığı için, projenin minimum Python 3.8 (veya üzeri) ile çalıştırılması gerekir.



PERFORMANS ÖLÇÜTLERİ

Bir API'nin performansını ölçmek için kullanılan temel metrikler şunlardır:

****Latency (Gecikme):**** Bir isteğin yanıt alması için geçen süre.

****Throughput (Verim):**** Birim zamanda işlenen istek sayısı.

Asenkron programlama, özellikle yüksek eşzamanlılık durumlarında Verimi önemli ölçüde artırır.



Eğer 'async with httpx.AsyncClient()' kullanılmazsa, istemci nesnesi manuel olarak kapatılmalıdır:

```
'client = httpx.AsyncClient()'
```

```
... işlemler ...
```

```
'await client.aclose()'
```

Ancak 'async with' bu işlemi otomatik yaptığı için tercih edilen yöntemdir.

COLAB ALIŞTIRMASI: YANIT KONTROLÜ

'requests' veya 'httpx' yanıt nesneleri, veriye farklı şekillerde erişim sağlar:
'response.text': Yanıtın ham dize (string) içeriği.
'response.json()': Yanıtın JSON olduğunu varsayarak onu otomatik olarak Python sözlüğüne dönüştürür.
JSON ayrıştırma hatası varsa 'response.json()' çağrısı istisna fırlatır.

GELİŞMİŞ GÜVENLİK: SSL/TLS DOĞRULAMASI

Harici API'lere istek yaparken, varsayılan olarak SSL/TLS sertifikası doğrulanır. Bu, iletişim kurulan sunucunun gerçekten iddia ettiği sunucu olduğunu garanti eder.

****UYARI:**** Bu doğrulamayı kapatmak ('verify=False' kullanmak) ciddi güvenlik riskleri oluşturur ve yalnızca test amaçlı kullanılmalıdır.



Daha karmaşık senaryoda, IoC'nin tipine göre farklı harici API'leri sorgulayabiliriz.

Eğer `'ioc.type == 'domain'` ise Whois API'sine git.

Eğer `'ioc.type == 'file_hash'` ise VirusTotal API'sine git.

Bu, 'if/elif' yapısını genişletmeyi gerektirir, böylece tek bir endpoint birden fazla harici servisi yönetebilir.

KODLAMA İNCELEMESİ: PYDANTIC MODEL_DUMP()

'ioc.model_dump()' metodu, Pydantic 2.0+ sonrası versiyonlarda kullanılır (önceki versiyonlarda 'dict()' idi).

Bu metot, Pydantic modelindeki tüm alanları ve değerleri, SQLAlchemy modeline doğrudan aktarılabilen bir Python sözlüğüne dönüştürür. Bu, veri aktarımını son derece hızlı ve tip güvenli hale getirir.



Veritabanı oturumu ('Session'), 'Depends(get_db)' aracılığıyla endpoint'e enjekte edilir.

FastAPI'deki bağımlılık sistemi, oturumun her istek için doğru şekilde açılmasını ve istek tamamlandığında otomatik olarak kapatılmasını sağlar (Genellikle try...finally blokları içerir).



SENARYO GENİŞLEMESİ: TEHDİT PUANI

Bazı tehdit API'leri, bir varlık için sayısal bir risk puanı (score) sağlar. Bu sayısal veriyi Indicator modeline bir 'threat_score: Integer' alanına kaydetmek, analistlerin hızlı kararlar almasını sağlar. Bu durumda, harici API'den gelen verinin tip kontrolü ve sayıya dönüştürülmesi gerekebilir.



ASYNCIO VE GIL (GLOBAL INTERPRETER LOCK)

Python'daki GIL, aynı anda yalnızca bir Python iş parçacığının CPU'yu kullanabileceği anlamına gelir.

Ancak, GIL ****G/Ç işlemleri sırasında serbest bırakılır****.

'httpx"ın asenkron ağ çağrılarını, bekleme süresi boyunca GIL'i serbest bıraktığı için, Python'un GIL kısıtlamasına rağmen yüksek eşzamanlılığa olanak tanır.

HARİCİ API BAĞIMLILIĞI RİSKLERİ

Uygulamanız harici bir API'ye çok bağımlı hale gelirse, o API'nin kesintisi sizin uygulamanızı da durdurabilir.

Bu riskleri azaltmak için ****Circuit Breaker (Devre Kesici)**** desenleri kullanılır. Devre Kesici, hatalı bir API'ye belirli bir süre boyunca istek göndermeyi durdurarak sistemin diğer kısımlarını korur.



COLAB PRATIĞİ: VERİ TİPİ DÖNÜŞÜMÜ

Harici API'den gelen sayısal veya boolean veriler bile bazen dize (string) olarak gelebilir.

Öğrenciler, bu verileri kendi Python kodlarında kullanmadan önce doğru tiplere (`int()`, `bool()`) dönüştürme pratiği yapmalıdır.

Örneğin, `'int(api_data['count'])'`.

HTTX VE KISA SÜRELİ İSTEKLER

Çok nadiren harici istek yapacaksanız ve 'AsyncClient' yönetiminin ek yükünü istemiyorsanız, 'httpx.get()' gibi basit bir asenkron fonksiyon da kullanabilirsiniz.

```
'response = await httpx.get(url)'
```

Ancak bu, 'AsyncClient'in bağlantı havuzlama avantajlarını sağlamaz. Sürekli API etkileşimi için 'AsyncClient' idealdir.

SONUÇ: KURALLARIN HATIRLATILMASI

1. Asenkron kod kullanırken daima 'await' kullanmayı unutmayın (I/O-bound işlemlerde).
2. Hata yönetimi (try/except) zorunludur.
3. Hassas bilgileri (API Key) asla kod içine gömmeyin. Ortam değişkeni kullanın.
4. FastAPI'nin asenkron doğasına uyan istemciler (httpx) tercih edin.



İLERİ OKUMA: ASYNC İÇİN FARKLI KÜTÜPHANELER

Python ekosisteminde asenkron HTTP istekleri için başka seçenekler de mevcuttur:

1. 'aiohttp': Özellikle WebSockets ve düşük seviyeli ağ işlemleri için güçlüdür.
2. 'urllib3' (artık çoğu kütüphane bunu temel alır): Daha düşük seviyeli bağlantı yönetimi sunar.

Ancak FastAPI topluluğunda, kullanım kolaylığı ve senkron/asenkron desteği nedeniyle 'httpx' öne çıkar.

Bu hafta, modern API geliştirme pratiğinde sadece kendi kodumuzu değil, harici servisleri de nasıl güvenli, hızlı ve dayanıklı bir şekilde entegre edebileceğimizi öğrendik.

Entegrasyon, mikroservislerin temelini oluşturur ve asenkron programlama bu entegrasyonun kilit performans faktörüdür.

Güvenlik (credential management) her zaman öncelikli olmalıdır.



API'DEN API'YE İLETİŞİM

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

